Mansoura University
Faculty of Computers and Information
Department of Information Technology
Second Semester- 2025-2026

[UNI311T]Software Reengineering

Grade: 4 SW

BY: Hala Ahmed

Outline

- Legacy Systems
- Impact Analysis
- Refactoring
- Program Comprehension
- Software Reuse

Legacy System

- **A legacy system** is **an old system** which **is valuable for the company** which often **developed and owns it**.
- It is the phase out stage of the software evolution model of **Rajlich and Bennet** described earlier.
- **Bennett** used the following definition:

> **"large software systems that we don't know how to cope with but that are vital to our organization."**

- Similarly, **Brodie and Stonebraker**: **define a legacy system as**

> **"any information system that significantly resists modification and evolution to meet new and constantly changing business requirements."**

Legacy System

- There are a number of options available to manage legacy systems.
- **Typical solution include**:
  - **Freeze:** The organization decides no further work on the legacy system should be performed.
  - **Outsource:** An organization may decide that supporting software is not core business and may outsource it to a specialist organization offering this service.
  - **Carry on maintenance:** Despite all the problems of support, the organization decides to carry on maintenance for another period.

Legacy System(Cont.)

- **Discard and redevelop:** Throw all the software away and redevelop the application once again from scratch.

- **Wrap:** It is a black-box modernization technique – surrounds the legacy system with a software layer that hides the unwanted complexity of the existing data, individual programs, application systems, and interfaces with the new interfaces.

- **Migrate:** Legacy system migration basically moves an existing, operational system to a new platform, retaining the legacy system's functionality and causing minimal disruption to the existing operational business environment as possible.

Outline

- Legacy Systems
- **Impact Analysis**
- Refactoring
- Program Comprehension
- Software Reuse

Impact Analysis

- **Impact analysis** is the task of **estimating the parts of the software** that can be affected if a proposed change request is made.
- **Impact analysis** techniques can be **partitioned into two classes**:
  - **Traceability analysis :** In this approach **the high-level artifacts** such as requirements, design, code and test cases related to the feature to be changed are identified. A model of inter-artifacts such that each artifact in one level links to other artifacts is constructed, which helps to locate a pieces of design, code and test cases that need to be maintained.
  - **Dependency (or source-code) analysis:** Dependency analysis attempt to assess the affects of change on semantic dependencies between program entities. This is achieved by identifying the syntactic dependencies that may signal the presence of such semantic dependencies.
    - The **two dependency-based** impact analysis techniques are: call graph based analysis and dependency graph based analysis.

Impact Analysis

- **Two additional notions** related to impact analysis are very common among practitioners:
  - **Ripple effect.**
  - **Change propagation.**
- **Ripple effect analysis** measures the **impact**, or how likely it is that a change to a particular module may cause problems in the rest of the program.
- **Measuring ripple effect** can provide knowledge about the system as a whole through its evolution:
  - how much its **complexity has increased** or **decreased** since the previous version,
  - how complex individual parts of a system are in relation to other parts of the system, and
  - to look at the effect that a new module has on the complexity of a system as a whole when it is added.

- **The change propagation activity** ensures that **a change made in one component** is propagated properly throughout the entire system.

Outline

- Legacy Systems
- Impact Analysis
- **Refactoring**
- Program Comprehension
- Software Reuse

Refactoring

- **Refactoring** is the process of making **a change to the internal structure of software** to make it **easier to understand** and **cheaper to modify** **without changing its observable behavior.**
- It is the object-oriented equivalent of **restructuring.**
- **Refactoring**, which aims to improve the internal structure of the code, achieve through the removal of duplicate code, simplification, making code easier to understand, help to find defects and adding flexibility to program faster.
- **There are two aspects of the above definition:**
  - It must preserve the "**observable behavior**" of the software system (through regression).
  - To improve the **internal structure** of a software system (improve maintainability).

Outline

- Legacy Systems
- Impact Analysis
- Refactoring
- **Program Comprehension**
- Software Reuse

Program Comprehension

- **Program understanding or comprehension** " is the task **of building mental models** of an underlying software system at various **abstraction levels**, **ranging from models** of the code itself to ones of the underlying application domain, for software maintenance, evolution and re-engineering purposes."
- **A mental model** describes **a programmer's mental representation of the program** to be comprehended.
- Program comprehension deals with **the cognitive processes** involved in **constructing a mental model** of the program.

Program Comprehension(Cont.)

- **A common element of such cognitive models** is generating hypotheses and investigating whether they hold or must be rejected.
  - **Hypotheses** are a way to understand code in an incremental manner.
  - **After some understanding of the code**, the programmer forms a hypothesis and verifies it by reading code.
  - **By continuously formulating new hypotheses and verifying** them, the programmer understands more and more code and in increasing details.

Program Comprehension(Cont.)

- **Several strategies** can be used to arrive at relevant hypotheses such as:
  - **bottom up** (starting from the code).
  - **top down** (starting from high-level goal).
  - **opportunistic combinations of the two.**
- **A strategy** is formulated by identifying actions to achieve a goal.

Program Comprehension(Cont.)

- **Strategies guide two mechanisms**, namely **chunking** and **cross-referencing** to produce higher-level abstraction structures.
  - **Chunking** creates new, higher level abstraction structures from lower-level structures.
  - **Cross-referencing** means being able to make links elements of different abstraction levels.
- **Chunking and cross-referencing** helps in building mental model of the program under study at different levels of abstractions.

Outline

- Legacy Systems
- Impact Analysis
- Refactoring
- Program Comprehension
- **Software Reuse**

Software Reuse

- **Software reuse** was introduced by Dough McIlroy in his 1968 seminal paper:

  "The development of an industry of **reusable source-code software components** and the industrialization of the production of application software from off-the-shelf components."

- **Other significant early reuse research developments include**:
  - **Program families** (David Parnas).
  - **Domain analysis** (Jim Neighbors).

Software Reuse(Cont.)

- **Program families** are **sets of programs** whose common properties are **so extensive** that it becomes advantageous to study the common properties of these programs before analyzing individual differences.
- Whereas **domain analysis** is **an activity of identifying objects and operations** of a class of similar systems in a particular problem domain.

Software Reuse(Cont.)

- **Software reuse** is using **existing artifacts** or **software knowledge** during the **construction of a new software system.**
  - **Reusable assets** can be either **reusable artifacts** or **software knowledge**.
- **Capers Jones** identified **four types of reusable artifacts**:
  - **data reuse,** involving a standardization of data formats,
  - **architectural reuse**, which consists of standardizing a set of design and programming conventions dealing with the logical organization of software,
  - **design reuse,** for some common business applications, and
  - **program reuse**, which deals with reusing executable code.

Software Reuse(Cont.)

- **Software reuse** of previously written code is a **way to increase:**
  - **software development productivity**.
  - **quality of the software.**
- The cost savings during maintenance as a consequence of reuse are nearly twice the corresponding savings during development.

Software Reuse(Cont.)

- Reusability is a property of a software assets that indicates the degree to which it can be reused.
- For software component to be reusable, it must exhibit the following characteristics that directly encourage its use is similar situation:
  - **Environmental independence -** The components can be reused irrespective of the environment from which they were originally captured.
  - **High cohesion** - The components that implement a single operation or set of related operations.
  - **Loose coupling -** The components that have minimal links to other components.

Software Reuse(Cont.)

- **Adaptability -** The components that are adaptable so they can be customized to fit a range of similar situation.
- **Understandability -** The components which are easily understandable that users can quickly interpret functionality.
- **Reliability -** The components that are error-free.
- **Portability -** The components that are not restricted in terms of the software or hardware environment they operate in.

Software Reuse(Cont.)

Benefits of Reuse

- Increased reliability.
- Reduced process risk.
- Increase productivity.
- Standards compliance.
- Accelerated development.
- Improve maintainability.
- Reduction in maintenance time and effort.

Chapter 2

Taxonomy Of Software Maintenance And Evolution

Outline

- General Idea
  - Intention-based Classification of S/W Maintenance
  - Activity-based Classification of S/W Maintenance
  - Evidence-based Classification of S/W Maintenance
- Categories of Maintenance Concepts
  - Maintained Product
  - Maintenance Types
  - Maintenance Organization Process
  - Peopleware

Outline

- Evolution of Software Systems
  - SPE Taxonomy
  - Laws of Software Evolution
  - Empirical Studies
  - Practical Implications of the Laws
- Maintenance of COTS-based Systems
  - Why Maintenance of CBS is Difficult?
  - Maintenance Activities for CBSs
  - Design Properties of Component-based Systems

General Idea

- **In circa 1972**, R. G. Canning in his landmark article "**The Maintenance 'Iceberg',**" discussed the problems of software maintenance.
- Practitioners took a narrow view of maintenance as
  - **correcting errors**, and
  - **enhancing the functionalities of the software.**

General Idea(Cont.)

- **The ISO/IEC 14764 standard** defines software maintenance as

  "... **the totality of activities** required to provide **cost-effective support** to a software system. Activities are performed during the **pre-delivery stage** as well as **the post-delivery stage**."

  - **Post-delivery activities** includes changing software, providing training, and operating a help desk.
  - **Pre-delivery activities** include planning for post-delivery operations.

Intention-based Classification Of Software Maintenance

- **Corrective maintenance:** The purpose of corrective maintenance is to **correct failures**: **processing failures** and **performance failures**.
- **Examples of corrective maintenance:**
  - A program producing a wrong output is an example of processing failure.
  - Similarly, a program not being able to meet real-time requirements is an example of performance failure.
- **The process of corrective maintenance** includes **isolation** and **correction** of defective elements in the software.
- **There is a variety of situations** that can be described as **corrective maintenance** such as **correcting a program** that aborts or **produces incorrect results**.
- Basically, **corrective maintenance** is a **reactive process**, which means that corrective maintenance is performed **after detecting defects with the system**.

Intention-based Classification Of Software Maintenance(Cont.)

- **Adaptive maintenance:** The purpose of adaptive maintenance is to enable the system to adapt to changes in its data environment or processing environment.
- This process modifies the software to properly interface with a changing or changed environment.
- **Adaptive maintenance** includes **system changes**, **additions**, **deletions**, **modifications, extensions, and enhancements** to meet the evolving needs of the environment in which the system must operate.
- **Examples of Adaptive maintenance are**:
  - changing the system to support new hardware configuration;
  - converting the system from batch to on-line operation; and
  - changing the system to be compatible with other applications.

Intention-based Classification Of Software Maintenance(Cont.)

- **Perfective maintenance:** The purpose of perfective maintenance is to make a variety of improvements, namely, user experience, processing efficiency, and maintainability.
- **Examples of perfective maintenance are:**
  - the program outputs can be made more readable for better user experience;
  - the program can be modified to make it faster, thereby increasing the processing efficiency;
  - and the program can be restructured to improve its readability, thereby increasing its maintainability.
- **Activities for perfective maintenance** include **restructuring of the code**, **creating** and **updating documentations**, and **tuning the system** to improve performance.
- It is also called "**reengineering**".

Intention-based Classification Of Software Maintenance(Cont.)

- **Preventive maintenance:** The purpose of preventive maintenance is to prevent problems from occurring by modifying software products.
- **Preventive maintenance** is very often performed on safety critical and high available software systems.
- The concept of **"software rejuvenation"** is a preventive maintenance measure to prevent, or at least postpone, the occurrences of failures (crash) due to continuously running the software system.
- It involves occasionally terminating an application or a system, cleaning its internal state, and restarting it.
- **Rejuvenation** may **increase the downtime** of the application; however, it prevents the occurrence of more severe failures.

Activity-based Classification of Software Maintenance

- **Kitchenham et al.** organize maintenance modification activities **into two categories**:
  - **Corrections:** Activities in this category are designed to fix defects in the system, where a defect is a discrepancy between the expected behavior and the actual behavior of the system.
  - **Enhancements:** Activities in this category are designed to effect changes to the system. It is further divided into **three subcategories as follows**:
    - **enhancement activities that modify** some of the existing requirements implemented by the system;
    - **enhancement activities that add new** system requirements; and
    - **enhancement activities that modify the implementation** without changing the requirements implemented by the system.

Evidence-based Classification of Software Maintenance

- **Twelve types of maintenance activities** were grouped into **four clusters**.
- Modifications performed, detected, or observed on four aspects of the system being maintained are used as the criteria to cluster the types of maintenance activities:
  - the whole software;
  - the external documentation;
  - the properties of the program code; and
  - the system functionality experienced by the customer.

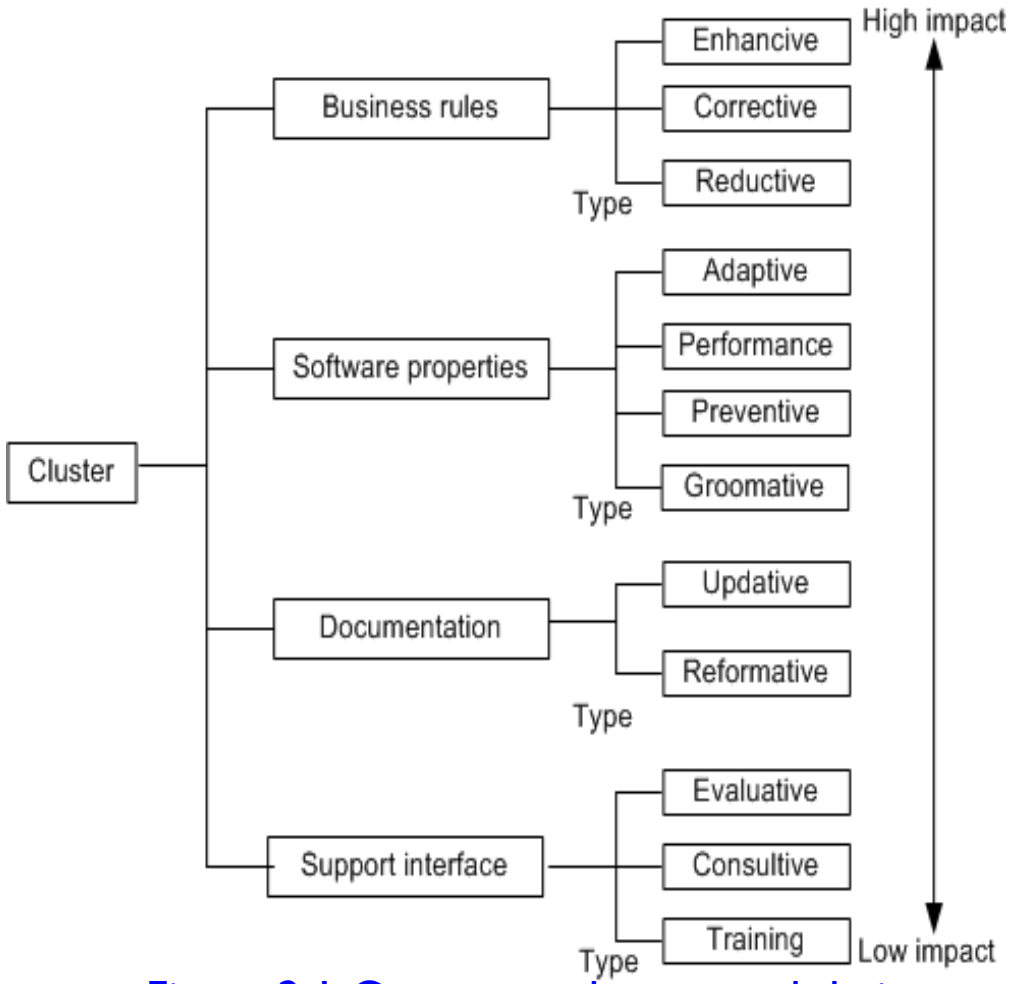

Figure 2.1 Groups or clusters and their types

Evidence-based Classification of Software Maintenance(Cont.)

- **Classification of maintenance activities** is based on changes in the **four kinds** of entities
  - the whole software;
  - the external documentation;
  - the properties of the program code; and
  - the system functionality experienced by the customer
- **Evidence of changes** to those entities is gathered **by comparing the appropriate portions of the software** **before** the activity with the appropriate parts **after** the execution of the activity.

Evidence-based Classification of Software Maintenance(Cont.)

- **Training:** This means **training the stakeholders** about the implementation of the system.
- **Consultive:** In this type, **cost and length of time are estimated** for maintenance work, personnel run a help desk, customers are assisted to prepare maintenance work requests, and personnel make expert knowledge about the available resources and the system to others in the organization to improve efficiency.
- **Evaluative:** In this type, common activities include **reviewing the program code** and **documentations**, **examining the ripple effect** of a proposed change, **designing** and **executing tests**, **examining the programming support** provided by the operating system, and finding the required data and debugging.

Evidence-based Classification of Software Maintenance(Cont.)

- **Reformative:** **Ordinary activities** in this type **improve the readability of the documentation**, make the **documentation consistent** with other changes in the system, prepare training materials, and add entries to a data dictionary.

- **Updative:** **Ordinary activities** in this type are **substituting out-of-date documentation** with up-to-date documentation, **making semi-formal**, say, in UML to document current program code, and updating the documentation with test plans.

- **Groomative:** Ordinary activities in this type are substituting components and algorithms with more efficient and simpler ones, modifying the conventions for naming data, changing access authorizations, compiling source code, and doing backups.

Evidence-based Classification of Software Maintenance(Cont.)

- **Preventive:** Ordinary activities in this type perform changes to **enhance maintainability and establish a base for making a future transition** to an emerging technology.

- **Performance:** Activities in performance type **produce results that impact the user**. Those activities improve system up time and replace components and algorithms with faster ones.

- **Adaptive:** Ordinary activities in this type port the software to a different **execution platform**, and **increase the utilization of COTS components**.

Evidence-based Classification of Software Maintenance(Cont.)

- **Reductive:** Ordinary activities in this type **drop some data generated** for the **customer**, **decreasing the amount of data input to the system**, and **decreasing the amount of data produced by the system**.

- **Corrective:** Ordinary activities in this type are **correcting identified bugs**, **adding defensive programming strategies**, and **modifying the ways exceptions** are handled.

- **Enhancive:** Ordinary activities in this type are adding and modifying business rules to enhance the system's functionality available to the customer, and adding new data flows into or out of the software.

Evidence-based Classification of Software Maintenance(Cont.)

- The impacts of the different types of maintenance activities are summarized in Table 2.2.
- The first dimension is the customer's ability to perform its business function while continuing to use the system. This is arranged based on the # of diamonds.
- The second dimension is the software. This is arranged from top to bottom.

Impact on software	Impact on business Low← – – – – – – → High	Cluster and type
Low		**Support interface**
↑	◇ ◇ ◇ ◇ ◇	Training
|	◇ ◇ ◇ ◇	Consultive
|	◇ ◇	Evaluative
|		**Documentation**
|	◇ ◇	Reformative
|	◇ ◇	Updative
|		**Software properties**
|	◇ ◇	Groomative
|	◇ ◇ ◇	Preventive
|	◇ ◇ ◇	Performance
|	◇ ◇	Adaptive
|		**Business rules**
|	◇	Reductive
|	◇ ◇ ◇	Corrective
↓	◇ ◇ ◇ ◇ ◇ ◇	Enhancive
High		

Table 2.2: Impact of the types [15] (©[2001] John Wiley)

Evidence-based Classification of Software Maintenance(Cont.)

Decision Tree Based Criteria

- Activities are classified into different types by applying **a two-step decision process**:
  - First, apply criteria-based decisions to make the clusters of types.
  - Next, apply the type decisions to identify one type within the cluster.

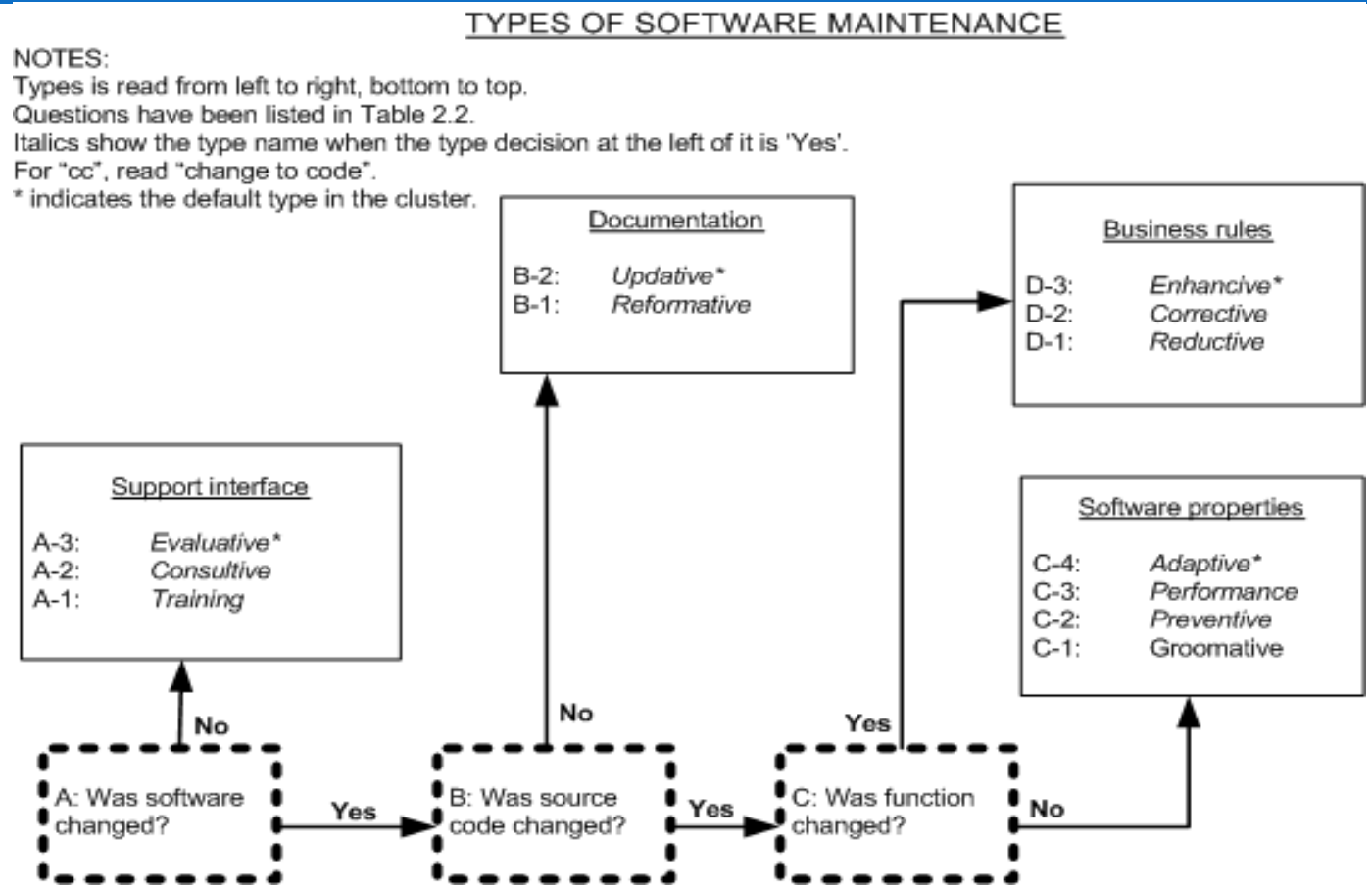

Figure 2.1 Decision Tree Types

Evidence-based Classification of Software Maintenance(Cont.)

- Sometimes, an objective evidence may be found to be ambiguous. In that case, clusters have their designated default types for use. The overall default type is evaluative, if there are ambiguities in an activity.

Criteria	Type decision question	Type
A-1	To train the stakeholders, did the activities utilize the software as subject?	Training
A-2	As a basis for consultation, did the activities employ the software?	Consultive
A-3	Did the activities evaluate the software?	Evaluative
B-1	To meet stakeholder needs, did the activities modify the non-code documentation?	Reformative
B-2	To conform to implementation, did the activities modify the non-code documentation?	Updative
C-1	Was maintainability or security changed by the activities?	Groomative
C-2	Did the activities constrain the scope of future maintenance activities?	Preventive
C-3	Were performance properties or characteristics modified by the activities?	Performance
C-4	Were different technology or resources used by the activities?	Adaptive
D-1	Did the activities constrain, reduce, or remove some functionalities experienced by the customer?	Reductive
D-2	Did the activities fix bugs in customer-experienced functionality?	Corrective
D-3	Did the activities substitute, add to, or expand some functionalities experienced by the customers?	Enhancive

Table 2.3: Summary of evidence-based types of software maintenance

Categories of Maintenance Concepts

The four categories and the concepts that influence the maintenance process have been illustrated in the following Fig. 2.3.

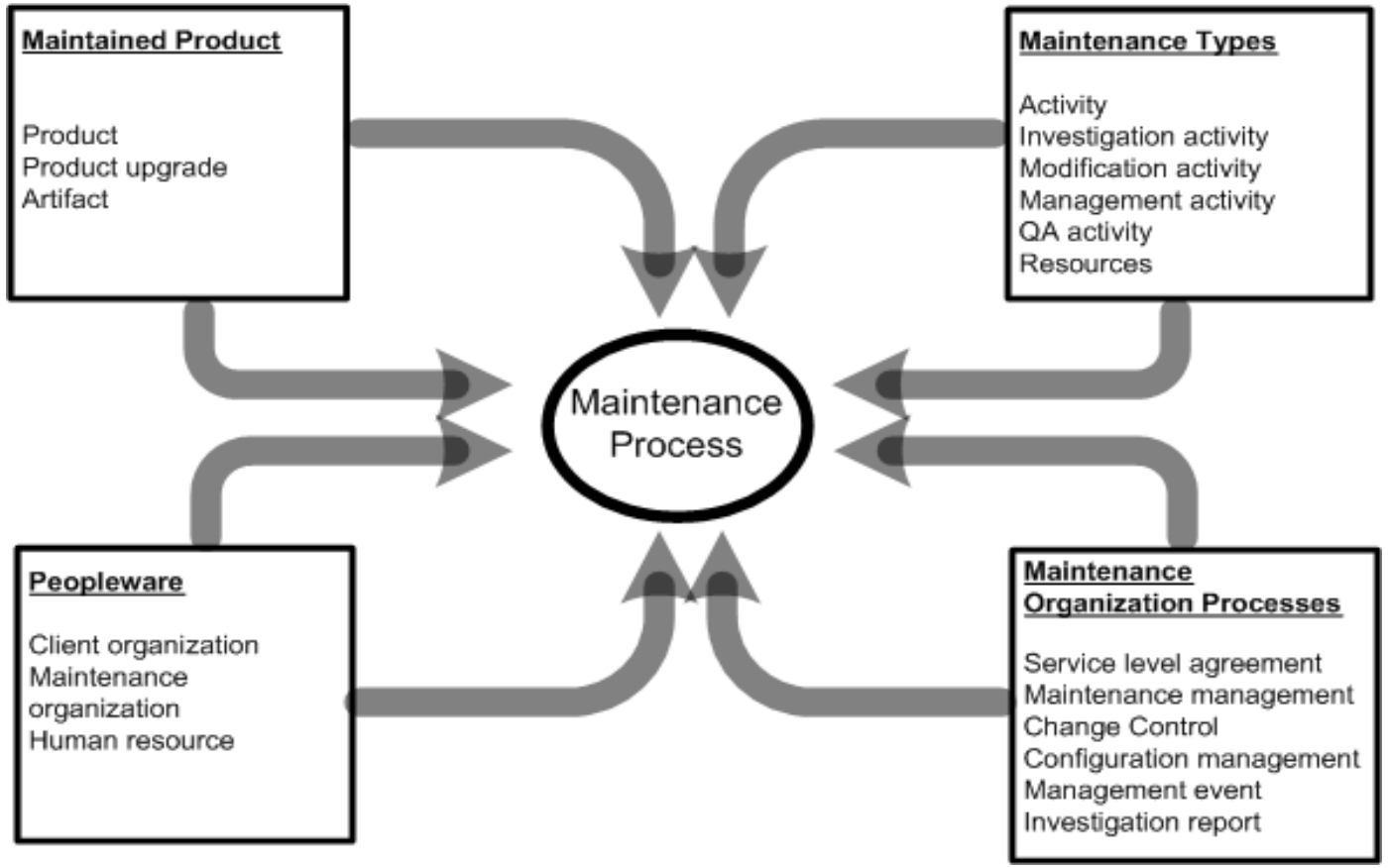

Figure 2.3 Overview of concept categories affecting software maintenance

Maintained Product

- **The maintained product dimension** is characterized by **three concepts**:
  - **Product:** A product is a coherent collection of several different artifacts. Source code is the key component of a software product.
  - **Product upgrade:** Baseline is an arrangement of related entities that make up a particular software configuration. Any change or upgrade made to a software product relates to a specific baseline.
  - An upgrade can create a new version of the system being maintained, a patch code for an object, or even a notice explaining a restriction on the use of the system.

Maintained Product

- **Artifacts:** A number of different artifacts are used in the design of a software product. One can find the following types of artifacts: textual and graphical documents, component off-the-shelf products, and object code components.
  - The key elements of the maintained product are size, age, application type, composition and quality.

TABLE 2.4 Staged Model Maintenance Task

Life Cycle Stage	Maintenance Task	User Population
Initial development	–	–
Evolution	Corrections, enhancements	Small
Servicing	Corrections	Growing
Phaseout	Corrections	Maximum
Closedown	Corrections	Declining

Maintenance Types

- **Four types of maintenance activities are defined**:
  - **Investigation activity:** This kind of activities evaluate the impact of making a certain change to the system.
  - **Modification activity:** This kind of activities change the system's implementation.
  - **Management activity:** This kind of activities relate to the configuration control of the maintained system or the management of the maintenance process.
  - **Quality assurance activity:** This kind of activities ensure that modifications performed on a system do not damage the integrity of the system.
- **Activity:** An activity accepts some artifacts as inputs and produces new or changed artifacts.
- **Artifacts:** Artifacts include plans, documents, system representations, and source and object code items. Artifacts are created during the software development process and changed during maintenance.

Maintenance Organization Processes

- **Two different levels of maintenance** processes are followed within a maintenance organization:
  - **Individual-level maintenance processes** followed by maintenance personnel to implement a specific change request, and
  - **Organization-level process** followed to manage the change requests from maintenance personnel, users, and customers/clients.

Maintenance Organization Processes(Cont.)

Three major elements of a maintenance organization are

- **Event management:** The stream of events, namely, all the change requests from various sources, received by the maintenance organization is handled in an event management process.

- **Configuration management:** A system's integrity is maintained by means of a configuration management process. Integrity of a product is maintained in terms of its modification status and version number.

- **Change control system:** Evaluation of results of investigations of maintenance events is performed in a process called change control. Based on the evaluation, the organization approves a system change.

Maintenance Organization Processes(Cont.)

- A maintenance organization handles maintenance change requests from:
  1. Users,
  2. Customers, and
  3. maintainers.

- After an initial investigation of a change request, a management process is put in place for approving change activities.

Maintenance Organization Processes(Cont.)

- Approval of a change request is normally the responsibility of a change control board.
- A proposed modification activity is scheduled only after the modification is approved by the board and an **Service Level Agreement (SLA)** is signed with the client.
- **Service level agreement (SLA)** is a contract between the **customers** and the **providers** of a maintenance service. Performance targets for the maintenance services are specified in the **SLA.**

Maintenance Organization Processes(Cont.)

In general, maintenance organizations use three different support levels to organize the staff:

- **Level 1:** This group files problem reports and identifies the technical support person who can best assist the person reporting a problem.
- **Level 2:** This level includes experts who know how to communicate with users and analyze their problems. These people recommend quick fixes and temporary workarounds.
- **Level 3:** This level includes programmers who can perform actual changes to the product software.

Peopleware

- Maintenance activities cannot ignore the human element, because software production and maintenance are human intensive activities.
- The **three people-centric concepts** related to maintenance are as follows:
  1. **Maintenance organization:** This is the organization that maintains the product(s).
  2. **Client organization:** A client organization uses the maintained system.
  3. **Human resource:** Human resource includes personnel from the maintenance and client organizations.

Peopleware(Cont.)

- Finally, the following user and customer issues affect maintenance:

  - **Size:** The size of the customer base and the number of licenses they hold affect the amount of effort needed to support a system.
  - **Variability:** High variability in the customer base impacts the scope of maintenance tasks.
  - **Common goals:** The extent to which the users and the customer have common goals affect the SLAs.
    - Ultimately, customers fund maintenance activities.
    - If the customers do not have a good understanding of the requirements of the actual users, some SLAs may not be appropriate to the end users.

Thank you